

河海大学计算机与软件学院

《数据库系统原理实践》

openGauss 场景综合作业 设计

组长学号

姓名

同组同学

学年学期

专业年级

指导老师



目录

1	项目概述	4
1.1	项目背景	4
1.2	设计目标	4
2	需求分析	4
2.1	业务需求分析	4
2.1.1	核心业务场景	4
2.1.2	业务流程要求	5
2.2	功能需求分析	5
2.2.1	用户与身份管理	5
2.2.2	一卡通生命周期管理	6
2.2.3	卡片状态流转	6
2.2.4	商户与消费终端管理	6
2.2.5	资金交易处理	7
2.3	数据流图	7
2.3.1	0 层数据流图	7
2.3.2	1 层数据流图	8
2.3.3	2 层数据流图	9
2.4	数据结构	10
3	数据库设计	12
3.1	概念设计	12
3.1.1	局部 E-R 模型构建	12
3.1.2	全局 E-R 图设计与基数约束分析	13
3.2	逻辑设计	14
3.2.1	逻辑图设计	14
3.2.2	关系模式规范化	14
3.2.3	逻辑结构清单下	16
3.3	物理设计	16
3.3.1	索引优化策略	16
3.3.2	视图抽象与封装	16
3.4	数据库定义	17
3.4.1	DDL 语句	17
3.4.2	数据填充	19
3.4.3	触发器定义	20
3.5	数据库操作与查询	25
3.5.1	基础检索	25
3.5.2	聚合与统计	26
3.5.3	复杂逻辑连接	27

3.5.4	高级分析	30
3.6	视图与索引管理	33
3.6.1	视图管理	33
3.6.2	索引管理	33
3.7	用户与权限管理	34
3.7.1	用户管理与权限管理	34
4	JDBC 连接与 GUI 设计	35
4.1	JDBC 连接	35
4.2	GUI 界面设计	37
5	实验总结	38
5.1	项目回顾	38
5.2	技术收获	38
5.2.1	数据库设计能力提升	38
5.2.2	数据库编程与优化	38
5.2.3	安全与权限管理	38
5.3	组员个人总结	38
5.3.1	陈俊仁个人总结	错误!未定义书签。
5.3.2	张厚冰个人总结	错误!未定义书签。
5.3.3	杨佳伟个人总结	错误!未定义书签。
6	团队分工	38
7	附录	39
7.1	附录 A: 个人作业	39
7.1.1	陈俊仁个人作业	错误!未定义书签。
7.1.2	张厚冰个人作业	错误!未定义书签。
7.1.3	杨佳伟个人作业	错误!未定义书签。
7.2	附录 B: JDBC 代码	39

1 项目概述

1.1 项目背景

某高校为实现校园消费与管理的数字化、便捷化，计划建设校园一卡通消费系统。需对该系统的数据库进行设计。系统核心管理对象包括学生、教职工、校园一卡通卡片、消费商户、消费记录、充值记录。

系统关系说明：用户需办理校园一卡通卡片作为消费凭证，卡片分为学生卡、教职工卡等类型；用户可在校园内指定商户，如食堂、超市、书店、开水房等进行消费；用户可通过线下充值点或线上渠道为一卡通卡片充值；系统需记录每笔消费明细和充值明细，便于用户查询、学校对账及账务管理。

1.2 设计目标

本系统旨在构建一套全流程的校园一卡通消费数字化管理平台，以实现从用户办卡、资金充值、多场景消费到账务查询的业务闭环。系统需全面覆盖学生与教职工的身份管理，支持食堂、超市等多元化商户业态的接入，并能够精确记录每笔消费与充值明细，从而满足用户实时查询、学校财务对账及审计的精细化管理需求，确保校园消费数据的实时性与准确性。

在核心实体与业务规则设计上，系统严格遵循规范化建模标准。用户管理实行实名制，包含姓名、身份证号等关键信息，并确立“一人一主卡”的对应关系，支持卡片在正常、挂失、补办等状态间的流转与历史追溯。商户管理则涵盖编号、类型、位置及营业状态的维护，确保交易场景的有效性。

为保障数据的完整性与资金安全，系统将充分利用数据库的主键、外键及检查约束机制，防止数据冗余与非法操作。特别是在核心资金处理层面，系统需建立严格的事务控制逻辑，确保余额基于“累计充值减去累计消费”的公式实时更新，并实施余额非负的强制约束，从而实现交易的可追溯性与数据的一致性。

2 需求分析

2.1 业务需求分析

本系统的业务逻辑围绕“用户—卡片—商户”这一生态闭环展开，旨在构建一个集身份认证、资金流转与信息服务于一体的综合管理平台。通过对核心业务场景的抽象与流程规范化，确保系统在复杂交互下的稳定性与数据一致性。

2.1.1 核心业务场景

核心业务场景构成了系统的功能骨架，主要涵盖身份构建、资金流通与信息追溯三个核心维度的交互逻辑。具体如下：

- 身份认证与准入体系。**系统需构建严格的准入机制以确立信任基础。在持卡端，系统需区分学生与教职工两类主体，通过办理校园一卡通确立其在校园消费生态中的合法身份与消费凭证。在受理端，系统需支持商户注册并建立详细档案，从而形成用户持卡消费与商户受理交易的双向信任关系。
- 资金闭环流通体系。**资金流转是系统的核心脉搏，包含资金注入与流出两个方

向。充值业务作为资金入口，需支持线上电子支付与线下充值点等多渠道接入，确保资金即时且安全地注入账户。消费业务作为资金出口，要求持卡用户能在指定商户终端完成身份识别与即时扣款。系统需支撑高并发下的交易处理，保障每笔资金流动的准确无误。

3. 信息透明与追溯服务。为保障用户的知情权与系统的可审计性，系统需提供全维度的查询服务。用户不仅能够实时获取当前余额状态，更应能追溯每一笔消费与充值的历史明细。这不仅实现了账务的透明化管理，也为学校财务部门的对账工作提供了数据支撑。

2.1.2 业务流程要求

业务流程的设计不仅是步骤的串联，更是对数据状态流转与事务一致性的严格约束。

1. 用户信息的全生命周期管理始于信息的采集与审核。用户提交基础身份信息后，系统需进行严格的逻辑校验与资格审核，审核通过后方可初始化账户并从物理层面发放一卡通。此流程确立了系统数据的源头质量，确保了后续交易主体的合法性与唯一性。
2. 资金类操作的执行需严格遵循校验先行与原子更新的原则。在消费流程中，刷卡动作触发后系统必须优先验证卡片状态及余额充足性，校验通过后执行扣款、生成流水记录并更新余额，这多个步骤必须作为一个原子事务完成，以确保流水记录与账户余额的强一致性。充值流程同样需先验证卡片有效性，确认资金到账后同步记录流水并累加余额，防止虚假充值。
3. 卡片有效性的维护依赖于严谨的状态机管理机制。当发生挂失申请时，流程重点在于阻断风险，应立即冻结旧卡资金权限。在补办新卡环节，流程需完成旧卡注销与新卡初始化的无缝衔接，确保账户资金能安全继承至新卡，同时保留旧卡的历史记录以备追溯。
4. 商户运营活动的规范化依赖于状态的实时控制。除了基础信息的维护外，核心在于营业状态的设置与校验。该状态将直接作为消费交易的前置校验条件，确保处于停业或异常状态下的商户无法发起扣款请求，从而规范校园商业秩序并降低交易风险。

2.2 功能需求分析

2.2.1 用户与身份管理

2.2.1.1. 身份管理机制

需求	具体内容
基础信息采集	记录用户真实姓名、身份证号、联系方式及电子邮箱
唯一性约束	身份证号与联系方式需确保全局唯一，防止重复注册
身份分类标识	区分为"学生"与"教职工"两类，为后续差异化服务提供依据
数据完整性	核心身份信息为必填项，邮箱为选填

2.2.1.2. 安全认证机制

密码安全存储：采用哈希加密存储用户登录密码，确保敏感信息安全

2.2.2 一卡通生命周期管理

卡片作为资金载体与消费凭证，需实现全生命周期的精细化管理。

2.2.2.1. 卡片与用户关系管理

需求	具体内容
主卡唯一性	在任意时间点，每个用户仅能持有一张状态为"正常"的主用卡
历史卡片追溯	补办新卡时，原卡片状态由"挂失"转换为"注销"，保留历史记录
卡片状态维护	记录每张卡片的办理时间以及关联用户信息

2.2.2.2. 余额管理与状态控制

需求	具体内容
余额实时计算	卡片余额=累计充值金额-累计消费金额
性能优化设计	在卡片表维护"当前余额"字段，支持高频交易的快速读取
事务一致性保证	通过数据库事务机制确保余额与交易流水的一致性
余额约束	余额不得为负值，消费前需进行余额校验

2.2.3 卡片状态流转

a.状态定义：正常、挂失、注销三种状态

b.状态流转规则：

正常→挂失：用户申请挂失

挂失→正常：用户找回卡片

挂失→注销：用户申请补办新卡

注销状态为最终状态，不可恢复

2.2.4 商户与消费终端管理

商户是消费行为发生的物理场所，需进行统一管理与运营控制。

2.2.4.1. 商户基础信息管理

需求	具体内容
商户档案建立	记录商户编号、名称、经营类型、地理位置
类型分类管理	支持食堂、超市、书店、开水房等多种商户类型分类
位置信息维护	详细记录商户所在楼宇、楼层等位置信息，便于用户查找

2.2.4.2. 商户运营状态控制

需求	具体内容
状态定义	营业、停业两种运营状态
状态控制机制	停业状态下商户无法进行任何交易操作
状态变更权限	仅系统管理员有权变更商户营业状态
状态实时生效	状态变更立即生效，无需等待系统重启

2.2.5 资金交易处理

交易流水记录系统所有资金变动，需确保数据完整性与业务合规性。

2.2.5.1. 交易数据完整性要求

需求	具体内容
消费记录字段	消费流水号、卡号、商户号、消费金额、交易时间
充值记录字段	充值流水号、卡号、充值金额、充值方式、交易时间、交易状态
流水号生成规则	采用全局唯一序列生成，确保交易可追溯

2.2.5.2. 业务规则约束

需求	具体内容
金额正数约束	所有涉及金额的字段必须大于零
余额校验机制	消费时需实时校验卡内余额，防止超额消费
状态控制	仅正常状态的卡片可进行消费，仅成功状态的充值计入余额
商户状态校验	消费时需校验商户营业状态

2.2.5.3. 事务安全保证

需求	具体内容
原子性操作	每笔交易作为一个完整的事务单元
一致性维护	交易成功需同时更新流水记录和卡片余额
异常处理机制	交易失败时自动回滚，保持数据一致性

2.3 数据流图

2.3.1 0 层数据流图

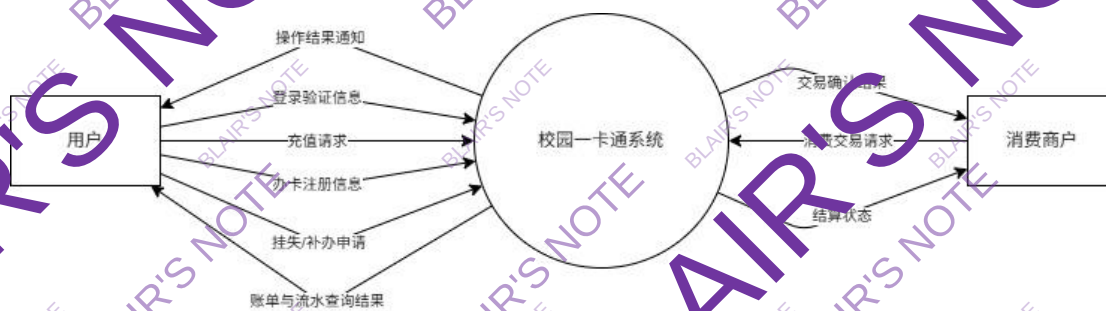


图1 零层数据流图

核心实体：用户（学生/教职工），校园一卡通系统，消费商户

主要数据流：

数据流向	含义
用户→系统	登录验证、充值、办卡、挂失、查询
系统→用户	操作结果、账单查询结果
商户→系统	消费交易请求
系统→商户	结算状态、交易确认结果

2.3.2 1层数据流图

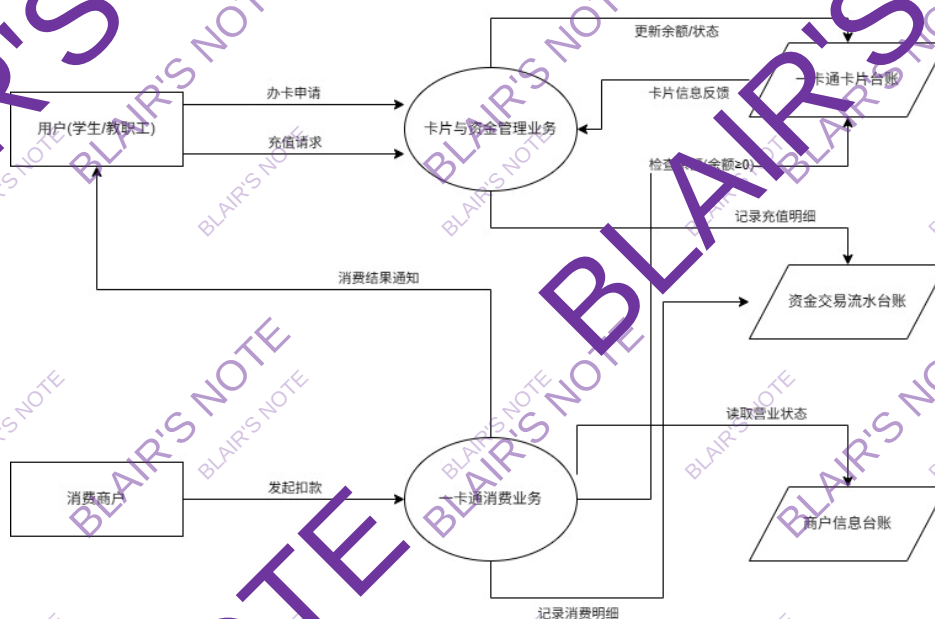


图2 一层数据流图

1. 用户（学生/教职工）：系统的使用者，包括学生和教职工，可发起“一卡通办卡申请”和“充值请求”。
2. 办卡申请：用户申请新的一卡通业务，既包括首次办卡，也包括补办卡。
3. 充值请求：用户向一卡通账户发起的充值操作，属于卡片与资金管理业务的

一部分。

4. 卡片与资金管理业务：负责处理与一卡通卡片及资金相关的后台业务，包括更新卡片余额与状态、记录充值明细、校验余额（余额 ≥ 0 ）、反馈卡片信息以及维护一卡通卡片台账。
5. 消费商户：校园内的各类商家，可发起扣款请求，属于一卡通消费业务的参与方。
6. 一卡通消费业务：负责处理消费扣款的业务逻辑，包括读取商户营业状态、维护商户信息台账、资金交易流水台账，并记录消费明细。
7. 消费结果通知：消费完成后，系统向用户或商户返回处理结果，涉及资金交易流水台账与商户信息台账的更新与反馈。

2.3.3 2 层数据流程图

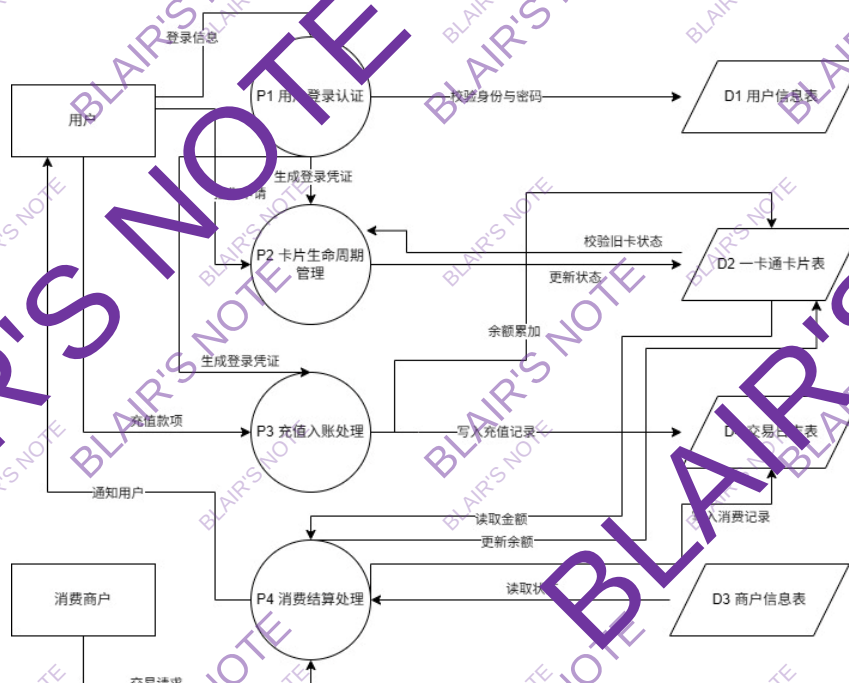


图 3 二层数据流程图

2.3.3.1. P1：用户登录认证

功能是验证用户身份。系统接收用户输入的账号和密码，访问 D1 用户信息表，根据账号查找对应用户记录，并将输入密码与数据库中存储的加密密码进行比对以完成身份校验。若校验成功，系统生成临时会话令牌，标志用户登录成功；在登录后的安全环境中，用户可进一步发起挂失申请，该数据流体现挂失操作需在完成身份认证后进行。

2.3.3.2. P2：卡片生命周期管理

功能是管理一卡通卡片从生成到失效的全过程，核心在于卡状态管理。系统接收挂失申请、充值款项以及办卡、补卡、注销等业务请求。对于挂失、补卡等操作，系

统首先校验卡片当前状态，确认卡片是否正常或是否已挂失，并根据业务类型在 D2 卡通卡片表中将状态更新为“挂失”“冻结”“补办新卡”或“注销”。对于充值请求，系统不直接入账，而是生成“余额累加”指令，将卡号与充值金额传递给 P3 模块完成具体的资金入账处理，同时向用户返回卡片信息及操作结果。

2.3.3.3. P3：充值入账处理

功能是安全、准确地完成资金向卡片余额的转移，并保证财务记录的可追溯性。系统接收来自 P2 的余额累加指令，首先在 D4 交易日志表中写入一条“充值”类型记录，包含卡号、时间、金额及操作员等关键信息，在完成日志记录后，再更新 D2 卡通卡片表中对应卡片的余额字段。充值完成后，系统通过页面提示、短信或 APP 消息等方式通知用户充值结果及最新余额。

2.3.3.4. P4：消费结算处理

功能是处理消费扣款请求，完成用户、商户与系统之间的资金结算。系统接收商户 POS 机发送的交易请求，请求中包含卡号、消费金额、商户 ID 和终端号等信息。系统首先访问 D3 商户信息表，校验商户是否处于营业状态以及终端是否有效；随后访问 D2 卡通卡片表，校验卡片状态是否正常，并读取卡片当前余额，判断余额是否大于或等于消费金额。若上述校验全部通过，系统在 D2 中扣减相应余额，并在 D4 中写入一条“消费”类型交易记录，记录卡号、商户信息、时间和金额等。处理结果将实时返回至 POS 机，消费结果通常通过小票或后续查询体现，在部分系统中也可进行实时通知；此外，系统还会在日终或批次结算时向商户提供汇总的结算清单，用于学校财务对账与资金划拨。

2.3.5. 整体链路

充值业务链路	用户→P2→P3→D4→D2→用户
消费业务链路	商户→P4→D3+D2→D2+D4→商户/用户
状态管理链路	P1→P2→D2→更新卡片状态

2.4 数据结构

1. 用户信息数据结构

数据项名	含义说明	数据类型	长度	取值范围	约束条件
UserID	用户唯一标识	Integer	4 bytes	> 0, 自增	Primary Key
UserName	用户真实姓名	Varchar	50	非空	Not Null
IDCardNo	身份证件号码	Char	18	符合国标	Unique, Not Null
Phone	移动联系电话	Varchar	20	数字	Unique, Not Null
Email	电子邮箱地址	Varchar	50	邮箱格式	Optional

UserType	用户身份类别	Varchar	20	Student/Teacher	Not Null
UserPassword	登录认证密码	Varchar	64	-	Not Null

2. 卡通卡片数据结构

数据项名	含义说明	数据类型	长度	取值范围	约束条件
CardNo	卡片物理卡号	Integer	4bytes	> 0, 自增	Primary Key
UserID	所属用户 ID	Integer	4 bytes	存在于用户表	Foreign Key
Status	卡片当前状态	Integer	1 byte	0:正常, 1:挂失, 2:注销	Default 0
Balance	账户当前余额	Numeric	(10,2)	≥ 0.00	Check (≥ 0)
CreateTime	开卡办卡时间	Timestamp	-	$\leq$ 当前时间	CURRENT_TIMESTAMP

3. 商户信息数据结构

数据项名	含义说明	数据类型	长度	取值范围	约束条件
MerchantID	商户内部编号	Integer	4 bytes	> 0, 自增	Primary Key
MerchantName	商户对外名称	Varchar	100	-	Not Null
MerchantType	商户经营类型	Varchar	30	-	Not Null
MerchantLocation	商户经营地址	Varchar	200	-	Optional
Status	营业状态标识	Varchar	10	Open/Closed	Default 'Open'

4. 消费记录数据结构

数据项名	含义说明	数据类型	长度	取值范围	约束条件
ConsumeID	消费流水编号	BigInt	8 bytes	> 0, 自增	Primary Key
CardNo	支付卡号	Varchar	20	存在于卡片表	Foreign Key

MerchantID	收款商户编号	Integer	4 bytes	存在于商户表	Foreign Key
Amount	本次交易金额	Numeric	(10,2)	> 0.00	Check (>0)
ConsumeTime	交易发生时间	Timestamp	-	-	CURRENT_TIME STAMP
Remark	交易附加备注	Varchar	200	-	Optional

5. 充值记录数据结构

数据项名	含义说明	数据类型	长度	取值范围	约束条件
RechargeID	充值流水编号	BigInt	8 bytes	> 0, 自增	Primary Key
CardNo	充值目标卡号	Varchar	20	存在于卡片表	Foreign Key
Amount	充值入账金额	Numeric	(10,2)	> 0.00	Check (>0)
Memod	资金来源渠道	Varchar	20	Online/Offline	Not Null
Status	充值结果状态	Varchar	20	Success/Fail	Default 'Success'
RechargeTime	充值发生时间	Timestamp	-	-	CURRENT_TIME STAMP

3 数据库设计

3.1 概念设计

3.1.1 局部 E-R 模型构建

本系统的概念设计基于核心业务对象进行抽象，主要包含用户、卡片及交易记录三大核心实体域。

3.1.1.1. 用户实体构建分析

用户实体构成了系统的核心主体，是所有业务行为的发起者。在建模过程中设计者遵循数据相关性原则，将自然人抽象为仅包含消费与管理相关属性的系统实体。UserID 作为唯一标识符确立了用户在系统中的全局唯一性。真实姓名、身份证号与移动电话等关键属性不仅用于严格的实名身份核验，也为后续的账户安全验证与即时通知服务提供了基础数据支撑。

3.1.1.2. 卡片实体构建分析

卡片实体作为资金存储与交易凭证的数字化载体，在逻辑模型中起到了连接用户身份与资金流水的枢纽作用。该实体以 CardNo 为唯一标识符，确立了物理卡片与系统账户的一一对应关系。除了记录办卡时间与卡片类型等基础静态信息外，该实体重点维护余额与状态两个核心动态属性，其中状态属性用于支持正常、挂失及注销等全生命周期的业务流转控制，确保资金状态的实时可控。

3.1.1.3. 交易动作的实体化策略

针对消费与充值等动态交互行为，本设计采用了联系实体化的建模策略。尽管在理论层面交易行为属于实体间的动作联系，但考虑到系统对每一笔资金变动均需进行持久化存储与精细化审计，设计者将消费记录与充值记录从单纯的关系提升为具备独立属性的实体。这种设计不仅解决了多维联系中属性挂载的复杂性，更为记录交易金额、时间戳及流水号等详细凭证提供了实体依托，从而满足了后续账务核对与历史追溯的严苛业务需求。

3.1.2 全局 E-R 图设计与基数约束分析

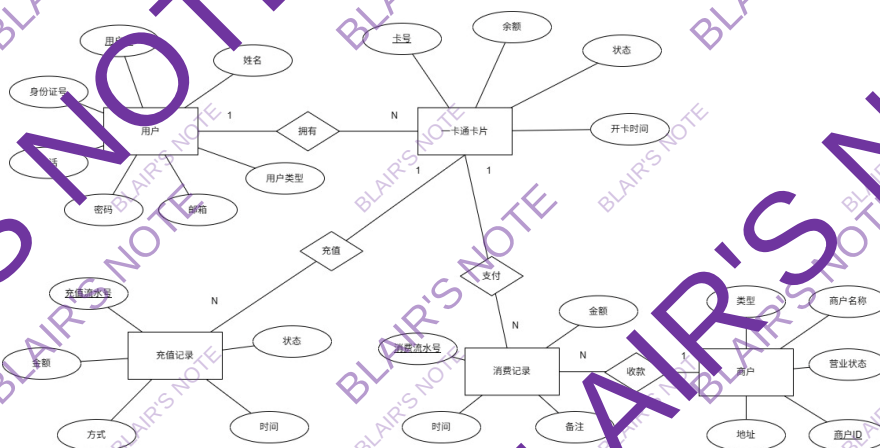


图 4-E-2 图

系统内各实体间通过明确的业务规则构建了严密的逻辑联系，具体的基数约束关系分析如下：

1. 用户与一卡通卡片之间存在一对多的联系。虽然在单一时间点上一个用户仅持有一张有效的主卡，但考虑到挂失补办及卡片换发等业务场景，系统需保留旧卡的历史状态记录以供审计。因此在数据库层面，一个用户实体可关联多条历史卡片记录，而任意一张特定的卡片实体在任意时刻仅归属于一个确定的用户，从而构成了严格的一对多归属关系。
2. 一卡通卡片与消费记录之间呈现一对多的联系。作为消费行为的发起载体，一张卡片在全生命周期内可产生多笔消费流水。反之，每一条消费记录实体必须包含明确的卡片标识符以关联至特定的支付账户，这种强依赖关系确保了每一笔资金扣减均有据可查，实现了交易的可追溯性。
3. 商户与消费记录之间同样遵循一对多的联系规则。商户作为交易的受理方，在其经营周期内会接收并处理海量的消费请求。而对于单笔消费记录而言，其必须且只能关联至唯一的商户实体，这种约束确立了资金流向的明确性，

为后续基于商户维度的账务结算与经营分析提供了数据基础。

4. 一卡通卡片与充值记录之间存在一对多的联系。为了维持卡片的持续消费能力，用户需对同一张卡片进行多次资金注入操作。系统通过将多条充值记录关联至同一张卡片，实现了对该卡片累计充值金额的准确统计与余额变化的精确追踪，确保了资金账目的完整性。

3.2 逻辑设计

3.2.1 逻辑图设计

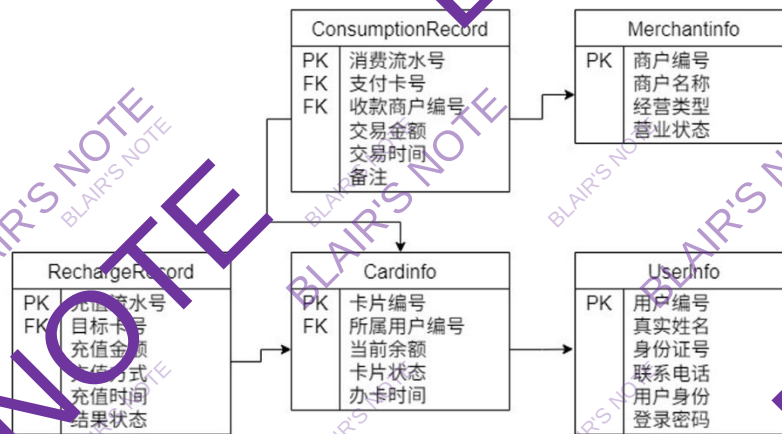


图 5 逻辑图

系统的逻辑结构设计遵循规范化原则，构建了以卡片信息表 CardInfo 为核心枢纽的关联模型，共包含五个标准化的实体关系表。

在该模型中，CardInfo 表处于连接用户身份与资金流水的中心位置，其通过外键 UserID 严格从属于用户信息表 UserInfo，从而确立了物理卡片与逻辑账户的唯一归属关系。针对高频发生的资金变动业务，设计方案采用了将资金流入与流出分离的策略，分别建立充值记录表 RechargeRecord 与消费记录表 ConsumptionRecord。这种设计不仅清晰了账务结构，有效隔离了不同性质的资金操作，更便于后续针对不同业务类型进行独立的索引优化与审计分析。

此外，消费记录表通过外键 MerchantID 与商户信息表 MerchantInfo 建立关联，有效闭合了由用户至卡片再至商户的完整交易链路。整个逻辑模型充分利用数据库引擎的主外键约束机制，在底层强制维护了数据的参照完整性，防止了孤立数据或无效交易记录的产生。

3.2.2 关系模式规范化

对于范式的证明，我举了 card_info 为例子，证明如下

```
R2 = Card_Info (R2)
U2 = {CardNo, UserID, Status, Balance, CreateTime}
F2 = {
    CardNo → UserID,
    CardNo → Status,
```

```

CardNo → Balance,
CardNo → CreateTime
}
候选码: K2 = {CardNo}

```

3.2.2.1. 第一范式(1NF)

定义: 表中的所有字段值都是不可再分的原子值。

验证: 对于 $\forall A \in U_2$: CardNo: 字符串域, UserID: 整数域, Status: {0,1,2}, Balance: 数值域, CreateTime: 时间戳域, 因此 $R_2 \in 1NF$

3.2.2.2. 第二范式(2NF)

定义: 在满足 1NF 基础上, 每一个非主属性完全函数依赖于 R 的码

验证: 主键: CardNo, 非主属性: UserID, Status, Balance, CreateTime 主键是单属性, 天然满足 2NF

3.2.2.3. 第三范式(3NF)

定义: 在满足 2NF 的基础上, 不存在非主属性对码的传递函数依赖。

验证: 函数依赖: $CardNo \rightarrow UserID, Status, Balance, CreateTime$ (直接依赖), $CardNo \rightarrow UserID \rightarrow Username$, username 不在本表中, 所以不是本表的传递依赖, Status、Balance、CreateTime 都直接依赖于 CardNo

3.2.2.4. 第四范式(4NF)

定义: 在满足 BCNF 的基础上, 消除非平凡的多值依赖。一个关系模式 R 属于 4NF, 如果对于 R 中的每个非平凡多值依赖 $X \twoheadrightarrow Y$, Y 不是 X 的子集, 且 $X \cup Y \neq R$, X 都是 R 的超键。

验证:

F_2 中所有非平凡函数依赖:

$CardNo \rightarrow UserID$

$CardNo \rightarrow Status$

$CardNo \rightarrow Balance$

$CardNo \rightarrow CreateTime$

每个依赖的左部 CardNo 都是候选码, 因此 $R_2 \in BCNF$ 。

反证法: 假设 $R_2 \notin 4NF$, 则存在非平凡多值依赖 $X \twoheadrightarrow Y$, 且 X 不包含候选码。

设 $X = \{UserID\}$, 需要 Y 满足: $Y \not\subseteq X$ 且 $X \cup Y \neq U_2$

可能的 Y:

1. $Y = \{CardNo\}$: 但 $CardNo \rightarrow$ 其他属性, 所以 $UserID \twoheadrightarrow CardNo$ 不成立
2. $Y = \{Status\}$: 一个用户的多张卡可能有不同状态, 不是多值依赖
3. $Y = \{Balance\}$: 不同卡余额不同, 不是多值依赖

4. $Y=\{CreateTime\}$: 不同卡开户时间不同, 不是多值依赖

$\therefore$ 不存在这样的 $X \rightarrow Y$

$\therefore R_2 \in 4NF$

3.2 逻辑结构清单下:

User_Info 表和 Card_Info 表通过 UserID 关联。

Card_Info 表和 Consume_Record 表通过 CardNo 关联。

Consume_Record 表和 Merchant_Info 表通过 MerchantID 关联。

Card_Info 表与 Recharge_Record 表通过 CardNo 关联。

3.3 物理设计

物理设计旨在将逻辑模型转化为具体的数据库存储结构, 以在存储效率与访问性能之间取得最佳平衡。本系统采用 openGauss 数据库引擎默认的堆表存储结构, 该结构对高并发的顺序写入操作具有天然的性能优势, 契合交易流水表数据即时写入的业务特性。

3.3.1 索引优化策略

针对系统读写分离的业务特征, 设计方案在保障写入性能的前提下, 重点通过建立辅助 B-Tree 索引来优化高频查询路径的响应速度。

3.3.1.1. 用户维度的复合索引设计

针对用户频繁查阅个人账单与最新交易记录的核心场景, 系统建立了包含卡号与消费时间的复合索引 `idx_consume_card_time`。该索引遵循最左前缀原则, 并特别针对时间字段采用了降序排列策略。这一设计具有多重性能收益: 首先, 它实现了覆盖索引扫描, 允许数据库直接从索引树中获取查询所需字段而无需回表读取数据页, 显著降低了 I/O 开销; 其次, 降序索引结构天然契合移动端的分页加载需求, 使得获取最新 N 条记录的操作能够利用索引有序性避免全量排序, 将查询的时间复杂度从线性扫描优化为对数级查找; 最后, 该索引高效支持了基于时间段的范围检索, 大幅减少了无效数据的扫描量。

3.3.1.2. 商户维度的结算索引设计

面向商户端的财务对账与报表生成需求, 系统部署了以商户编号为前导列的复合索引 `idx_consume_merchant_time`。该索引结构主要优化了按商户分组的聚合查询与特定时间窗内的范围扫描性能。在处理如商户日结、月结报表或异常交易监控等批量分析任务时, 该索引能够有效缩小数据扫描范围, 加速分组排序过程。例如在执行诸如贫困生餐补发放核查等复杂审计任务时, 该索引能快速定位特定商户在特定时间段内的交易明细, 确保了管理决策的时效性。

3.3.2 视图抽象与封装

为了降低应用程序与底层数据模型的耦合度, 系统引入了视图层作为数据访问的中间抽象。

复杂连接逻辑的封装。系统定义了名为 campus_card.v_user_consumption 的综合视图，该视图在数据库层内部封装了用户信息表、卡片信息表、消费记录表与商户信息表之间的四表连接逻辑。通过将复杂的 JOIN 操作固化为标准视图，系统不仅简化了上层应用的代码开发，还屏蔽了底层表结构的变更风险。在实际应用中，如辅导员进行困难学生消费结构分析或消费总额核查时，只需对该视图发起简单的单表查询即可获得包含用户姓名、卡号、商户名称及金额在内的完整业务视图，极大地提升了数据检索的便捷性与逻辑复用性。

```
CREATE OR REPLACE VIEW campus_card.v_user_consumption AS
SELECT user_info.username, user_info.idcardno, card_info.cardno, merchant_info.merchantname, consume_record.amount, consume_record.consumetime, consume_record.remark
FROM (((campus_card.user_info
JOIN campus_card.card_info ON ((user_info.userid = card_info.userid)))
JOIN campus_card.consume_record ON (((consume_record.cardno)::text = (card_info.cardno)::text)))
JOIN campus_card.merchant_info ON ((consume_record.merchantid = merchant_info.merchantid)));
```

3.4 数据库定义

本系统定义了一卡通管理系统数据库，包含 **5 个核心表**和 **1 个视图**：

User_Info	用户身份信息表
Card_Info	一卡通卡片信息表
Merchant_Info	商户信息表
Consume_Record	消费交易记录表
Recharge_Record	充值记录表
v_user_consumption	用户消费明细视图

3.4.1 DDL 语句

```
-- 创建用户表
CREATE TABLE User_Info (
    UserID SERIAL PRIMARY KEY,
    UserName VARCHAR(50) NOT NULL,
    IDCardNo VARCHAR(20) NOT NULL UNIQUE,
    Phone VARCHAR(20) NOT NULL UNIQUE,
    Email VARCHAR(50),
    UserType VARCHAR(20) NOT NULL
    CHECK (UserType IN ('Student', 'Teacher')),
    Password VARCHAR(64) NOT NULL
);

-- 创建卡片表
CREATE TABLE Card_Info (
```

```
CardNo VARCHAR(20) PRIMARY KEY,  
UserID INT NOT NULL REFERENCES User_Info(UserID) ON DELETE CASCADE,  
Status INT DEFAULT 0,  
CHECK (Status IN (0, 1, 2)),  
Balance NUMERIC(10, 2) DEFAULT 0.00,  
CHECK (Balance >= 0),  
CreateTime TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- 创建商户表

```
CREATE TABLE Merchant_Info (  
MerchantID SERIAL PRIMARY KEY,  
MerchantName VARCHAR(100) NOT NULL,  
MerchantType VARCHAR(30) NOT NULL,  
MerchantLocation VARCHAR(200),  
Status VARCHAR(10) DEFAULT 'Open'  
CHECK (Status IN ('Open', 'Closed'))  
);
```

-- 创建消费记录表

```
CREATE TABLE Consume_Record (  
ConsumeID BIGSERIAL PRIMARY KEY,  
CardNo VARCHAR(20) NOT NULL  
REFERENCES card_info(CardNo) ON DELETE CASCADE,  
MerchantID INT NOT NULL  
REFERENCES merchant_info(MerchantID) ON DELETE CASCADE,  
Amount NUMERIC(10, 2) NOT NULL  
CHECK (Amount > 0.00),  
ConsumeTime TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
Remark VARCHAR(200)  
);
```

-- 创建充值记录表

```
CREATE TABLE Recharge_Record (  
RechargeID BIGSERIAL PRIMARY KEY,  
CardNo VARCHAR(20) NOT NULL  
REFERENCES card_info(CardNo) ON DELETE CASCADE,  
Amount NUMERIC(10, 2) NOT NULL  
CHECK (Amount > 0.00),  
Method VARCHAR(20) NOT NULL
```

```
CHECK (checked IN ('Online', 'Offline')),
Status VARCHAR(20) DEFAULT 'Success'
CHECK (Status IN ('Success', 'Fail')),
RechargeTime TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

3.4.2 数据填充

在这里我们各选取了具有代表性的五条数据：

用户信息样本：

UserID	UserName	IDCardNo	Phone	UserType	Email
1	张三	11010120000101101X	13800138001	Student	zhangsan@edu.cn
2	李四	110101198505052022	13900139002	Teacher	lisi@edu.cn
15	沈七	110101200106061156	18500185015	Student	shenqi@edu.cn
22	何四	110101197701011224	17700177022	Teacher	hesi@edu.cn
30	金二	110101198909091302	19900199030	Teacher	jiner@edu.cn

卡片信息样：

CardNo	UserID	Status	Balance	CreateTime	状态说明
C1001	1	0	156.00	2024-09-12	正常使用
C1015	15	0	245.50	2024-03-12	余额最高
C1022	22	1	0.00	2024-01-20	已挂失
C1007	7	0	500.00	2025-05-18	余额 500 元
C9022	22	2	10.00	2021-11-30	已注销

商户信息样本：

MerchantID	MerchantName	MerchantLocation	MerchantType	Status	MerchantID
1001	第一食堂一楼基本伙	第一食堂 1F	餐饮	Open	1001
1004	学子园超市	学子园 B1	零售	Open	1004
1006	校内咖啡厅	图书馆 1F	餐饮	Open	1006
1011	清真食堂	第二食堂2F	餐饮	Open	1011
1013	健身房中心	体育馆 B1	服务	Open	1013

消费记录样本:

ConsumeID	CardNo	MerchantID	Amount	ConsumeTime	消费说明
1	C1015	1001	15.50	2025-01-10 07:30:00	早餐消费
33	C1012	1015	120.00	2025-02-10 15:00:00	药品消费
20	C1024	1013	100.00	2025-01-28 19:00:00	健身消费
9	C1025	1004	55.20	2025-01-17 19:20:00	超市购物
7	C1019	1006	35.00	2025-01-15 15:30:00	咖啡消费

充值记录样本:

RechargeID	CardNo	Amount	Method	Status	RechargeTime
1	C1001	200.00	Online	Success	2025-01-01 09:00:00
7	C1012	500.00	Online	Success	2025-01-01 13:25:40
23	C1024	500.00	Online	Success	2025-02-04 12:00:00
11	C1005	50.00	Offline	Success	2025-01-15 15:30:00
40	C1028	200.00	Online	Success	2026-01-05 09:20:00

3.4.3 触发器定义

系统采用数据库触发器技术来实现复杂的业务规则约束与自动化数据处理，以下是四个核心触发器的定义及其逻辑分析。

3.4.3.1. 卡片状态自动失效触发器

```
CREATE OR REPLACE FUNCTION campus_card.auto_deactivate_lost_cards_func()
```

```
RETURNS trigger
```

```
LANGUAGE plpgsql
```

```
NOT FENCED NOT SHIPPABLE -- PostgreSQL 内部优化参数，通常保持默认
```

```
AS $$
```

```
BEGIN
```

```
-- 检查新插入的卡片是否为正常状态
```

```
IF NEW.cardstatus = 0 THEN
```

```
-- 将该用户的所有挂失卡更新为失效状态
```

```
-- 条件说明:
```

```

-- 1. userid 相同，同一用户
-- 2. cardstatus = 1：当前是挂失状态
-- 3. card_id <> NEW.cardid：排除新卡自身
UPDATE campus_card.card_info
SET cardstatus = 2
WHERE userid = NEW.userid
AND cardstatus = 1
AND cardid <> NEW.cardid;
END IF;
-- 返回 NEW 记录，允许插入操作继续
RETURN NEW;
END;
$$;

```

针对卡片补办业务中可能出现的凭证冲突问题，系统部署了基于插入事件的前置触发器。该机制旨在强制执行单一有效凭证的业务规则，即在任意时刻一个用户仅能持有一张正常状态的主卡。当系统识别到新的正常状态卡片即将写入数据库时，触发器会原子性地扫描该用户关联的所有历史记录，并将处于挂失状态的旧卡片立即流转为终态注销。这一自动化处理逻辑不仅实现了新旧卡片生命周期的无缝衔接，更从底层数据约束上根除了因补办流程未闭环而导致的一人多卡或双重有效凭证的安全隐患，从而保障了账户体系的严谨性。

3.4.3.2. 余额自动转移触发器

```

CREATE OR REPLACE FUNCTION campus_card.transfer_balance_func()
RETURNS TRIGGER AS -- 缺少：AS $$ 和 LANGUAGE plpgsql
DECLARE
    new_card_no VARCHAR; -- 用于存储目标正常卡的卡号
BEGIN
    -- 仅当卡片状态从挂失变为失效时触发
    IF OLD.status = 1 AND NEW.status = 2 THEN
        -- 查找同一用户的其他正常卡
        -- LIMIT 1：只取第一张正常卡，如果有多个正常卡则选择第一个
        SELECT cardno INTO new_card_no
        FROM campus_card.card_info
        WHERE userid = OLD.userid AND status = 0
        LIMIT 1;

        -- 如果找到正常卡，执行余额转移
        IF new_card_no IS NOT NULL THEN
            -- 将旧卡余额加到正常卡上
            UPDATE campus_card.card_info

```

```

SET balance = balance + OLD.balance

WHERE card_no = new_card_no;

-- 将失效卡的余额设为0
NEW.balance := 0;

END IF;

-- 注意：如果没找到正常卡，余额不会转移，但状态已变为失效
END IF;

RETURN NEW; -- 返回修改后的记录

END; -- 缺少：$$ LANGUAGE plpgsql;

```

为了在卡片生命周期更替时保障用户资产的安全继承，系统实现了基于状态驱动的余额自动转移机制。该触发器严密监控卡片状态由挂失向注销流转的特定变迁路径，一旦状态变更被触发，系统即刻自动识别用户当前持有的合法主卡，并执行资金的原子性迁移操作。该逻辑将失效卡片内的剩余款项全额划转至新卡账户，同时将原卡余额清零。这种设计确保了资金权益在物理介质更替过程中的连续性，消除了因人工干预延迟或遗漏可能引发的资金沉淀风险，从数据库底层实现了财务数据的自动化闭环管理。

3.4.3.3. 消费事务处理触发器

```

CREATE OR REPLACE FUNCTION campus_card.process_consumption_func()
RETURNS trigger
LANGUAGE plpgsql
NOT FENCED NOT SHIPPABLE
AS $$
DECLARE

    v_balance DECIMAL(10,2); -- 当前卡片的余额
    card_status VARCHAR; -- 当前卡片的状态

BEGIN

    -- 获取当前卡片的余额和状态，用于后续验证
    SELECT balance, status INTO v_balance, card_status
    FROM campus_card.card_info
    WHERE cardno = NEW.cardno; -- NEW.cardno 是插入的消费记录中的卡号

    -- 检查卡片是否存在：如果查询没找到记录，触发异常
    IF NOT FOUND THEN
        RAISE EXCEPTION '错误：卡号 % 不存在，无法消费。', NEW.cardno;
    END IF;

    -- 卡片状态验证：只有正常状态的卡可以消费
    IF card_status != 0 THEN

```

```

RAISE EXCEPTION '错误：卡片 % 当前状态为"%，非正常状态无法消费。', NEW.cardno, card_status;
END IF;

-- 余额充足性验证：消费金额不能超过当前余额
IF v_balance < NEW.amount THEN

    RAISE EXCEPTION '错误：卡片 % 余额不足，交易已回滚。',
        NEW.cardno, v_balance, NEW.amount;
END IF;

-- 执行扣费操作：原子性更新卡片余额
UPDATE campus_card.card_info
SET balance = balance - NEW.amount
WHERE cardno = NEW.cardno;

-- 记录成功日志
RAISE NOTICE '消费成功：卡号 % 扣除 % 元，剩余余额 % 元。',
    NEW.cardno, NEW.amount, (v_balance - NEW.amount);

-- 返回 NEW 记录，允许消费记录插入
RETURN NEW;
END;

```

消费业务的资金安全性依赖于严格的前置校验与原子更新机制。本触发器作为消费交易的守门人，在记录写入前执行多重逻辑验证。系统首先校验物理卡片的存在性与状态合规性，确保仅有正常状态的卡片能发起交易。随后对账户余额进行充足性检查以从根本上杜绝透支现象。更为关键的是，该触发器将消费记录的生成与余额的扣减封装为一个不可分割的原子操作，确保了在并发环境下交易流水与账户余额的强一致性，有效防止了数据异常。

3.4.3.4. 充值事务处理触发器

```

CREATE OR REPLACE FUNCTION campus_card.process_recharge_func()
RETURNS trigger
LANGUAGE plpgsql
NOT FENCED NOT SHIPPABLE
AS $$
DECLARE
    v_balance DECIMAL(10,2); -- 当前卡片的余额
    card_status VARCHAR; -- 当前卡片的状态
BEGIN

```

```

-- 获取当前卡片的余额和状态，用于后续验证
SELECT balance, status INTO v_balance, card_status
FROM campus_card.card_info
WHERE cardno = NEW.cardno; -- NEW.cardno 是插入的充值记录中的卡号

-- 检查卡片是否存在
IF NOT FOUND THEN
    RAISE EXCEPTION '错误：卡号 % 不存在，无法充值。', NEW.cardno;
END IF;

-- 卡片状态验证：只有正常状态的卡可以充值
IF card_status != 0 THEN
    RAISE EXCEPTION '错误：卡片 % 当前状态为%', NEW.cardno, card_status;
END IF;

-- 执行充值操作：仅当充值记录状态为'Success'时才更新余额
-- 这样可以支持充值失败、处理中等中间状态
IF NEW.status = 'Success' THEN
    -- 更新卡片余额
    UPDATE campus_card.card_info
    SET balance = balance + NEW.amount
    WHERE cardno = NEW.cardno;

    -- 记录成功日志
    RAISE NOTICE '充值成功：卡号 % 存入 % 元，当前余额 % 元。',
        NEW.cardno, NEW.amount, (v_balance + NEW.amount);
ELSE
    -- 记录非成功状态的日志，余额不更新
    RAISE NOTICE '充值流水 % 状态为 %，未触发余额更新。', NEW.rechargeid, NEW.status;
END IF;

-- 返回 NEW 记录，允许充值记录插入
RETURN NEW;
END;
$$;
/

```

充值业务的处理逻辑重点在于资金来源的确认与状态过滤。该触发器在执行资金注入前，同样会对卡片的合法性与状态进行前置校验，防止向异常卡片注入资金。在更新逻辑上，系统引入了条件判断机制，仅当充值流水明确标记为成功状态时，才执

行余额的累加操作。这种设计有效隔离了支付失败或处理中的中间状态对账户余额的影响，确保了账务数据的准确性，实现了资金流入记录与账户增值操作的同步绑定。

3.5 数据库操作与查询

3.5.1 基础检索

3.5.1.1. 单表查询

查询所有状态为 'Open' 的商户

assignment=#**SELECT** merchantname, merchantlocation, merchanttype

assignment=#**FROM** campus_card.merchant_info

assignment=#**WHERE** status = 'Open';

merchantname	merchantlocation	merchanttype
第一食堂一楼基本伙	第一食堂 1F	餐饮
第一食堂二楼特色餐	第一食堂 2F	餐饮
第二食堂川菜档口	第二食堂 1F	餐饮
学子园超市	学子园 B1	零售
校园打印中心	教学楼 A 座 1F	服务
校内咖啡厅	图书馆 1F	餐饮
校园快递驿站	宿舍区东侧	服务
体育馆羽毛球馆	体育馆 1F	服务
自动贩卖机-综合楼	综合办公楼 2F	零售
阳光水果店	宿舍区商业街	零售
清真食堂	第二食堂 2F	餐饮
校园理发店	宿舍区商业街	服务
共享单车服务点	校门口	服务
校医院药店	校医院 1F	服务
健身房中心	体育馆 B1	服务

(15 rows)

3.5.1.2. 排序与限制

查询最近的 30 条流水记录

assignment=#**SELECT** consumeid, cardno, amount, consumetime**FROM** campus_card.consume_record **ORDER BY** consumetime **ASC** **LIMIT** 30;

consumeid	cardno	amount	consumetime
1 C1015		15.50	2025-01-10 07:30:00
2 C1003		45.00	2025-01-10 12:45:22
3 C1012		22.00	2025-01-11 18:40:10

4	C1007	18.00	2025-01-12 11:55:00
5	C1018	5.00	2025-01-13 14:20:45
6	C1001	12.50	2025-01-14 08:10:30
7	C1019	35.00	2025-01-15 15:30:00
8	C1005	12.80	2025-01-16 10:05:15
9	C1025	55.20	2025-01-17 19:20:00
10	C1008	40.00	2025-01-18 16:45:00
11	C1014	14.00	2025-01-19 07:45:33
12	C1004	25.00	2025-01-20 12:30:00
13	C1021	10.00	2025-01-21 09:15:00
14	C1010	68.00	2025-01-22 14:00:20
15	C1027	20.00	2025-01-23 17:10:00
16	C1018	19.50	2025-01-24 11:40:00
17	C1013	8.50	2025-01-25 20:15:00
18	C1026	42.00	2025-01-26 13:20:45
19	C1009	12.00	2025-01-27 08:00:00
20	C1024	100.00	2025-01-28 19:00:00
21	C1017	3.50	2025-01-29 10:30:15
22	C1023	24.00	2025-01-30 12:00:00
23	C1016	15.00	2025-01-31 15:50:00
24	C1029	16.50	2025-02-01 11:20:00
25	C1002	30.00	2025-02-02 09:00:00
26	C1022	12.00	2025-02-03 07:50:00
27	C1030	28.00	2025-02-04 18:30:00
28	C1011	15.00	2025-02-05 12:10:00
29	C1020	25.00	2025-02-06 14:40:00
30	C1006	45.00	2025-02-07 10:20:00

(30 rows)

3.5.2 聚合与统计

3.5.2.1. 简单聚合

统计系统内的总用户数

```
assignment=# SELECT COUNT(*) AS total_users FROM campus_card.user_info;
```

```
total_users
```

```
-----
```

```
30
```

3.5.2.2. 数值计算

查询某商户的平均单笔消费金额

```

assignment=# SELECT ROUND(AVG(amount), 2) AS avg_transaction FROM campus_card.consume_record WHERE merchantid = 1001;

avg_transaction
-----
13.75

```

3.5.2.3. 分组过滤

查询累计营收超过 100 元的商户

```

assignment=# SELECT m.merchantname, SUM(cr.amount) AS total_revenue
assignment=# FROM campus_card.consume_record cr
assignment=# JOIN campus_card.merchant_info m ON cr.merchantid = m.merchantid
assignment=# GROUP BY m.merchantname
assignment=# HAVING SUM(cr.amount) > 100
assignment=# ORDER BY total_revenue DESC;

merchantname | total_revenue
-----
校医院药店   | 181.00
健身房中心   | 180.00
学子园超市   | 172.00
校内咖啡厅   | 134.00
校园理发店   | 123.00
(5 rows)

```

3.5.2.4. 多表统计

统计每种用户类型的总持卡数量

```

assignment=# SELECT u.usertype, COUNT(c.cardno) as card_count
assignment=# FROM campus_card.user_info u
assignment=# JOIN campus_card.card_info c ON u.userid = c.userid
assignment=# GROUP BY u.usertype;

usertype | card_count
-----
Student  | 21
Teacher  | 24
(2 rows)

```

3.5.3 复杂逻辑连接

3.5.3.1 多表连接

```

assignment=# SELECT
assignment=# u.username,

```

```
assignment-# u.name,  
assignment-# c.cardno,  
assignment-# cr.amount,  
assignment-# cr.consumetime,  
assignment-# m.merchantname  
assignment-# FROM campus_card.user_info u  
assignment-# JOIN campus_card.card_info c ON u.userid = c.userid  
assignment-# JOIN campus_card.consume_record cr ON c.cardno = cr.cardno  
assignment-# JOIN campus_card.merchant_info m ON cr.merchantid = m.merchantid  
assignment-# LIMIT 10;
```

username	phone	cardno	amount	consumetime	merchantname
沈七	18500185015	C1015	15.50	2025-01-10 07:30:00	第一食堂一楼基本伙
王五	13700137003	C1006	45.00	2025-01-10 12:15:22	学子园超市
褚四	18800188012	C1012	22.00	2025-01-11 18:40:10	第一食堂二楼特色餐
吴九	13300133007	C1007	18.00	2025-01-12 11:55:00	清真食堂
严十	17100117028	C1028	5.00	2025-01-13 14:20:45	校园打印中心
张三	12800132001	C1001	12.50	2025-01-14 08:10:30	第二食堂川菜档口
秦一	18101181019	C1019	35.00	2025-01-15 15:30:00	校内咖啡厅
孙七	13500135005	C1005	12.80	2025-01-16 10:05:15	阳光水果店
张七	17400174025	C1025	55.20	2025-01-17 19:20:00	学子园超市
郑十	13200132008	C1008	40.00	2025-01-18 16:45:00	校园理发店

(10 rows)

3.5.3.2. 子查询

```
assignment-# SELECT * FROM campus_card.consume_record  
assignment-# WHERE amount > (SELECT AVG(amount) FROM campus_card.consume_record)  
assignment-# ORDER BY amount DESC;
```

consumeid	cardno	merchantid	amount	consumetime	remark
33	C1012	1015	20.00	2025-02-10 15:00:00	
20	C1024	1013	100.00	2025-01-28 19:00:00	
43	C1021	1013	80.00	2025-02-20 18:00:00	
14	C1010	1015	68.00	2025-01-22 14:00:20	
9	C1025	1004	55.20	2025-01-17 19:20:00	
28	C1005	1008	50.00	2025-02-15 16:00:00	
30	C1006	1012	45.00	2025-02-07 10:20:00	
2	C1003	1004	45.00	2025-01-10 12:15:22	
18	C1026	1006	42.00	2025-01-26 13:20:45	

10 C1008	1012	40.00	2025-01-18 16:45:00
50 C1024	1012	38.00	2025-02-27 15:20:00
7 C1019	1006	35.00	2025-01-15 15:30:00
40 C1008	1006	32.00	2025-02-17 14:10:00
25 C1002	1010	30.00	2025-02-02 09:00:00
27 C1030	1004	28.00	2025-02-04 18:30:00
(15 rows)			

3.5.3.3. 半连接

查询所有“有过消费记录”的用户且消费金额达到一定数额

```

assignment=# SELECT u.username, u.userid
assignment=# FROM campus_card.user_info u
assignment=# WHERE EXISTS (
assignment=# SELECT c
assignment=# FROM campus_card.card_info c
assignment=# JOIN campus_card.consume_record cr ON c.cardno = cr.cardno
assignment=# WHERE c.userid = u.userid AND cr.amount > (SELECT AVG(amount) FROM campus_card.
consume_record)
assignment(#);
username |userid
-----+-----
李四    |    2
王五    |    3
孙七    |    5
周八    |    6
郑十    |    8
冯二    |   10
褚四    |   12
秦一    |   19
许三    |   21
施六    |   24
张七    |   25
孔八    |   26
金二    |   30
(13 rows)

```

3.5.3.4. 反连接

查询“只办卡没充过值”的用户

```
assignment=# SELECT u.username
```

```

assignment=# FROM campus_card.user_info u
assignment=# JOIN campus_card.card_info c ON u.userid = c.userid
assignment=# WHERE NOT EXISTS (
assignment=# SELECT 1
assignment=# FROM campus_card.recharge_record r
assignment=# WHERE r.cardno = c.cardno
assignment(# );
username
-----

```

冯二

陈三

张三

秦一

沈七

孙七

郑十

褚四

赵六

金二

蒋六

何四

李四

尤二

张七

严十

陈三

(17 rows)

3.5.4 高级分析

3.5.4.1. 窗口函数

查询每张卡片的最近 1 笔消费

```

assignment=# SELECT * FROM (
assignment=# SELECT
assignment=# cardno,
assignment=# amount,
assignment=# consumetime,
assignment=# ROW_NUMBER() OVER (PARTITION BY cardno ORDER BY consumetime DESC) as
assignment=# rn
assignment=# FROM campus_card.consume_record

```

```

assignment(# r
assignment-# WHERE t.rm <= 2 LIMIT 10;
card_no | amount | consumetime | rm
-----+-----+-----+---
C1001 | 17.00 | 2025-02-13 11:30:00 | 1
C1001 | 12.50 | 2025-01-14 08:10:30 | 2
C1002 | 30.00 | 2025-02-02 09:00:00 | 1
C1003 | 22.50 | 2025-02-09 12:45:00 | 1
C1003 | 45.00 | 2025-01-10 12:15:22 | 2
C1004 | 26.00 | 2025-02-19 11:50:00 | 1
C1004 | 25.00 | 2025-01-20 12:30:00 | 2
C1005 | 50.00 | 2025-02-15 16:00:00 | 1
C1005 | 12.80 | 2025-01-16 10:05:15 | 2
C1006 | 45.00 | 2025-02-07 14:20:00 | 1
(10 rows)

```

3.5.4.2. 全外连接

每日资金流向报表

```

assignment-# SELECT
assignment-# COALESCE(r.day, c.day) as report_date,
assignment-# COALESCE(r.total_recharge, 0) as total_in,
assignment-# COALESCE(c.total_consume, 0) as total_out,
assignment-# (COALESCE(r.total_recharge, 0) - COALESCE(c.total_consume, 0)) as net_flow
assignment-# FROM (
assignment-# SELECT date_trunc('day', rechargetime) as day, SUM(amount) as total_recharge
assignment-# FROM campus_card.recharge_record GROUP BY 1
assignment-# ) r
assignment-# FULL OUTER JOIN (
assignment-# SELECT date_trunc('day', consumetime) as day, SUM(amount) as total_consume
assignment-# FROM campus_card.consume_record GROUP BY 1
assignment-# ) c ON r.day = c.day
assignment-# ORDER BY report_date DESC LIMIT 10;
report_date | total_in | total_out | net_flow
-----+-----+-----+-----
2026-01-05 00:00:00 | 200.00 | 0 | 200.00
2025-02-28 00:00:00 | 50.00 | 0 | 50.00
2025-02-27 00:00:00 | 100.00 | 38.00 | 62.00
2025-02-26 00:00:00 | 300.00 | 15.00 | 285.00
2025-02-25 00:00:00 | 100.00 | 5.50 | 94.50
2025-02-24 00:00:00 | 200.00 | 18.00 | 182.00

```

2025-02-23 00:00:00	0	21.00	-21.00
2025-02-22 00:00:00	150.00	15.00	135.00
2025-02-21 00:00:00	0	13.50	-13.50
2025-02-20 00:00:00	100.00	80.00	20.00

条件聚合

用户消费行为深度分析

```
assignment-# SELECT
assignment-#     u.username,
assignment-#     COUNT(cr.consumeid) as total_txns,
assignment-#     SUM(cr.amount) as total_spent,
assignment-#     MAX(cr.amount) as max_spent,
assignment-#     COUNT(CASE WHEN m.merchanttype = 'Canteen' THEN 1 END) as canteen_visits,
assignment-#     COUNT(CASE WHEN m.merchanttype = 'Supermarket' THEN 1 END) as market_visits
assignment-# FROM campus_card_user_info u
assignment-# JOIN campus_card_info c ON u.userid = c.userid
assignment-# LEFT JOIN campus_card_consume_record cr ON c.cardno = cr.cardno
assignment-# LEFT JOIN campus_card_merchant_info m ON cr.merchantid = m.merchantid
assignment-# GROUP BY u.userid, u.username
assignment-# ORDER BY total_spent DESC NULLS LAST
assignment-# LIMIT 20;
```

username	total_txns	total_spent	max_spent	canteen_visits	market_visits
----------	------------	-------------	-----------	----------------	---------------

褚四	2	142.00	120.00	0	0
施六	2	138.00	100.00	0	0
许三	2	90.00	80.00	0	0
冯二	2	81.50	68.00	0	0
郑十	2	72.00	40.00	0	0
王五	2	67.50	45.00	0	0
张七	2	64.00	55.20	0	0
孙七	2	62.80	50.00	0	0
秦一	2	54.00	35.00	0	0
赵六	2	51.00	26.00	0	0
孔八	2	47.50	42.00	0	0
周八	1	45.00	45.00	0	0
朱十	2	40.50	21.00	0	0
曹九	2	35.00	20.00	0	0
李四	1	30.00	30.00	0	0
吴九	2	30.00	18.00	0	0
张三	2	29.50	17.00	0	0

钱一	2	28.00	15.00	0	0
金二	1	28.00	28.00	0	0
卫三	2	26.50	18.00	0	0
(0 rows)					

3.6 视图与索引管理

3.6.1 视图管理

3.6.1.1. 创建视图

创建一个名为 v_student_consumes 的视图，展示学生的消费总额。

```
assignment=# CREATE VIEW v_student_consumes AS SELECT cardno, SUM(amount) FROM consume_record GROUP BY cardno;
```

CREATE VIEW

3.6.1.2. 修改视图

修改现有视图的定义，增加过滤条件。

```
assignment=# CREATE OR REPLACE VIEW v_student_consumes AS SELECT cardno, SUM(amount) FROM consume_record WHERE amount > 0 GROUP BY cardno;
```

CREATE VIEW

3.6.1.3. 使用视图进行查询

```
assignment=# SELECT * FROM v_student_consumes WHERE cardno = 'C1001';
```

cardno	sum
C1001	29.50

(1 row)

3.6.1.4. 重命名视图

将视图名称从 v_student_consumes 改为 v_summary。

```
assignment=# ALTER VIEW v_student_consumes RENAME TO v_summary;
```

ALTER VIEW

3.6.1.5. 删除视图

```
assignment=# DROP VIEW IF EXISTS v_summary;
```

DROP VIEW

3.6.2 索引管理

3.6.2.1 创建索引

在消费记录表的消费时间列上创建索引以加速查询。

```
assignment=# CREATE INDEX idx_time ON consume_record(consumetime);
CREATE INDEX
```

3.6.2.2. 重建索引

通过重新组织索引数据来优化性能，解决碎片问题。

```
assignment=# REINDEX INDEX idx_time;
REINDEX;
```

3.6.2.3. 重命名索引

将索引名称 idx_time 修改为 idx_consume_date。

```
assignment=# ALTER INDEX idx_time RENAME TO idx_consume_date;
ALTER INDEX
```

3.6.2.4. 删除索引

删除不再需要的索引以节省维护开销。

```
assignment=# DROP INDEX idx_consume_date;
DROP INDEX
```

3.7 用户与权限管理

3.7.1. 用户管理与权限管理

3.7.1.1. 创建用户并授权

```
-- PostgreSQL 用户创建与授权脚本
-- 功能：创建校园卡系统只读用户并分配权限
-- 1. 创建新用户 card_user，并设置密码为"Gauss#3campus"
CREATE USER card_user WITH PASSWORD 'Gauss#3campus';

-- 2. 授予用户对 campus_card 模式的使用权限
GRANT USAGE ON SCHEMA campus_card TO card_user;

-- 3. 授予用户对特定表的查询权限
GRANT SELECT ON TABLE
campus_card.card_info -- 校园卡基本信息表
campus_card.consume_record -- 消费记录表
TO card_user;
```

3.7.1.2. 切换用户并验证

1. 切换用户

```
assignment=# % assignment card_user # 连接到 assignment 数据库，使用 card_user 用户
```

```
Password for user card_user: # 输入 card_user 的密码
```

```
Non-SSL connection (SSL connection is recommended when requiring high-security)
```

```
You are now connected to database "assignment" as user "card_user". #连接成功提示
```

2. 测试查询

```
assignment=> SELECT * FROM campus_card.card_info LIMIT 5; # 查询校园卡信息表前5条记录
```

cardno	userid	status	balance	createtime
C1015	15	0	245.50	2024-03-12 00:00:00
C1022	22	1	0.00	2024-01-20 00:00:00
C1007	7	0	500.00	2025-05-18 00:00:00
C1012	12	0	88.60	2023-06-30 00:00:00
C1028	28	0	320.00	2023-12-01 00:00:00

(5 rows)

3. 测试查询未授权表(报错)

```
assignment=> SELECT * FROM campus_card.user_info LIMIT 5; # 尝试查询用户信息表
```

```
ERROR: permission denied for relation user_info
```

```
DETAIL: N/A
```

注意: card_user 可访问 card_info 表-用于业务操作

card_user 不可访问 user_info 表-保护敏感数据

4. JDBC 连接与 GUI 设计

4.1 JDBC 连接

JDBC (Java Database Connectivity) 是 Java 程序与数据库之间的标准接口，连接到校园卡数据库服务器。我们实现了 JDBC 连接，分别有 SQL 控制台、基本信息查看，卡片管理，商户管理，记录查询，业务办理等。在这里本文附上 JDBC 连接成功的输出并选取 GUI 的四个例图。

图 6 JDBC 连接成功



图 7. SQL 控制台



图 8 卡片管理

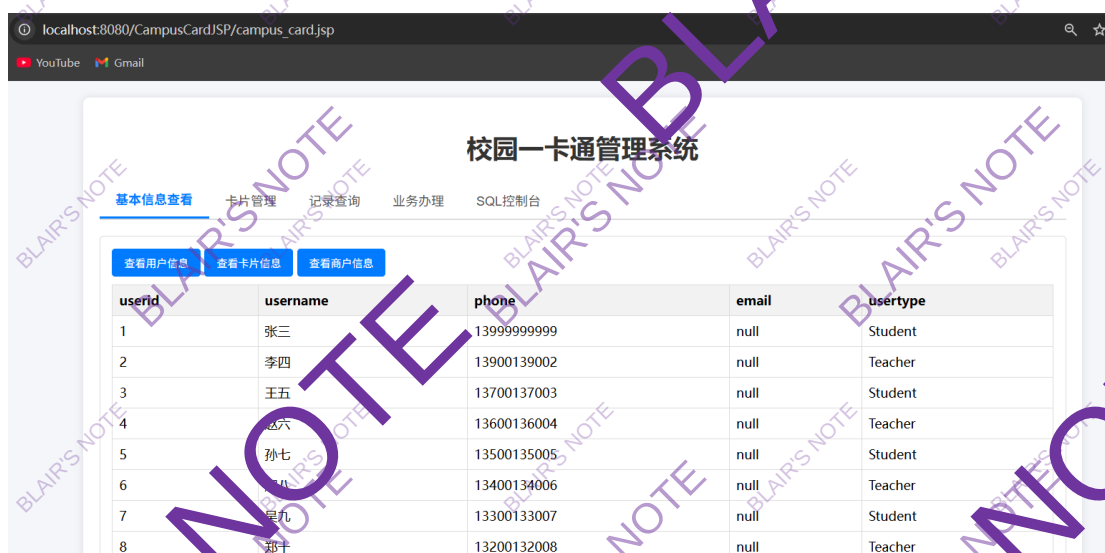


图 9 基本信息查看

type	time	target	amount
充值	2026-01-09 19:39:21.971881	Online	100.00
消费	2026-01-09 14:40:12.630639	校园超市	10.00
充值	2026-01-08 20:38:03.134219	Online	10.00
消费	2026-01-08 12:04:40.20097	第一食堂一楼基本伙	12.00
充值	2025-02-16 08:45:00.0	Offline	50.00
消费	2025-02-13 11:30:00.0	第一食堂二楼特色餐	17.00

图 10 充值或消费记录查看

4.2 GUI 界面设计



图 11 GUI 设计图

界面布局：采用深色科技风的三栏式结构。左侧是导航菜单，右侧是主要的操作显示区，底部是一个黑色的实时日志窗口。

核心功能：

1. 系统概览：

顶部有三个卡片显示核心指标：注册人数、总资金池、营业商户数。下方提供了一个 SQL 控制台，可以直接输入 SQL 语句查询数据库，结果会即时显示在表格中。

2. 数据监控 (用户/商户)：

提供了两张大表。“用户管理”查看全体用户的数据信息；“商户监控”查看所有店铺的信息。

3. 个人中心：

模拟普通用户的操作流程。包含一个登录界面，输入账号密码登录后，展示个人

详细资产，并通过下方的两个标签页分别查看个人的消费记录和充值流水。

4. 底部日志：

类似程序员的终端窗口。程序的每一步操作都会实时打印在这里，用于监控系统健康状态。

5 实验总结

通过本次《校园一卡通消费管理系统》数据库设计与开发实践，我们深入理解了数据库系统的设计流程、规范化理论、SQL 编程、性能优化与安全管理等内容，掌握了 openGauss 数据库的使用与开发方法。

5.1 项目回顾

本项目以校园一卡通消费系统为背景，围绕“用户—卡片—商户—消费—充值”五大核心业务，完成了从需求分析、概念设计、逻辑设计、物理设计到数据库实施、查询优化、权限管理及 GUI 应用开发的全流程实践。系统实现了用户注册、卡片管理、消费与充值、商户管理、资金流向统计等核心功能，并通过视图、索引、触发器、存储过程等技术手段提升了系统的完整性、安全性与性能。

5.2 技术收获

5.2.1 数据库设计能力提升

规范化理论应用：通过范式分析，确保数据结构的规范性与一致性，减少冗余与异常。

实体关系建模：掌握 E-R 图、逻辑图、数据流图的转换方法，明确实体、属性与联系的设计原则。

索引与视图设计：针对高频查询场景设计复合索引与视图，显著提升查询效率与开发便利性。

5.2.2 数据库编程与优化

触发器与事务控制：实现了消费、充值时的余额自动更新与状态控制，确保事务原子性与数据一致性。

复杂查询编写：熟练使用多表连接、子查询、窗口函数、条件聚合等高级 SQL 语法，支持复杂业务分析。

性能调优：通过索引优化、查询重构等手段，提升大数据量下的查询响应速度。

5.2.3 安全与权限管理

用户权限分级：通过角色与权限分离，实现数据访问控制，保护用户隐私与系统安全。

密码安全存储：采用哈希加密存储用户密码，增强系统安全性。

5.3 组员个人总结

6 团队分工

7 附录

7.1 附录 A：个人作业

7.2 附录 B：JDBC 代码

